Node.js — Day 5 Complete Notes

Topic: Node.js Modules — CommonJS basics

1. What is a Module?

A module is a separate piece of code that can be organized and reused in another file. Modules help with organization, reusability, and maintainability.

```
backend/
■■■ app.js
■■■ math.js
■■■ user.js
■■■ database.js
```

2. Exporting One Thing

module.exports makes a value or function available outside the module.

```
// math.js
function add(a, b) {
  return a + b;
}

module.exports = add;
```

```
// app.js
const add = require("./math");

console.log(add(5, 3)); // 8
```

require("./math") loads the local math.js module. **./** means the current folder.

3. Exporting Multiple Things

Multiple functions or values can be exported inside an object.

```
// math.js
function add(a, b) {
  return a + b;
}

function subtract(a, b) {
  return a - b;
}

module.exports = {
  add,
  subtract
};
```

```
// app.js
const math = require("./math");

console.log(math.add(10, 5));      // 15
console.log(math.subtract(10, 5)); // 5
```

4. Destructuring While Importing

Instead of importing the whole exported object, specific properties can be selected using JavaScript object destructuring.

```
const { add, subtract } = require("./math");
```

```
console.log(add(10, 5));
console.log(subtract(10, 5));
```

5. CommonJS Pattern

```
// Export one value
module.exports = add;

// Export multiple values
module.exports = {
  add,
  subtract
};

// Import a module
const math = require("./math");

// Import specific exports
const { add, subtract } = require("./math");
```

6. Why Modules Matter in Backend Development

As a backend grows, different responsibilities can live in different files. Later, we will use this idea with routes, controllers, services, database code, and utilities.

```
routes/
controllers/
services/
models/
utils/
server.js
```

7. Important Rules

• A module is a separate reusable piece of code.

• **module.exports** exports something from a CommonJS module.

• **require()** imports/loads a CommonJS module.

• **./** refers to the current folder.

• One value can be exported directly.

• Multiple values can be exported as an object.

• Destructuring can import specific properties from that exported object.

8. Practice Examples

```
// math.js
const multiply = (a, b) => a * b;
module.exports = multiply;

// app.js
const multiply = require("./math");
console.log(multiply(4, 5)); // 20
```

```
// math.js
const multiply = (a, b) => a * b;
const divide = (a, b) => a / b;

module.exports = { multiply, divide };

// app.js
```

```
const { multiply, divide } = require("./math");

console.log(multiply(6, 3)); // 18
console.log(divide(6, 3));   // 2
```

9. Quick Recap

Module: A separate reusable piece of code.

Export: Make a value/function available outside its file.

Import: Load a module into another file.

CommonJS export: module.exports

CommonJS import: require()

Multiple exports: Export an object and optionally destructure it while importing.

Key Takeaway

Modules let us split a Node.js application into small, reusable files instead of keeping all backend logic in one large file.